Republic of the Philippines
University of Southern Mindanao
College of Engineering and Information Technology
DEPARTMENT OF COMPUTING AND INFORMATION SCIENCE

HOMEWORK: K-Nearest Neighbors (KNN) on Diabetes Dataset

Members:
Tayba B. Abdullah
Charles Cabrera
Erika G Uyasan

Part 1: Data Understanding

1. Description of Each Feature

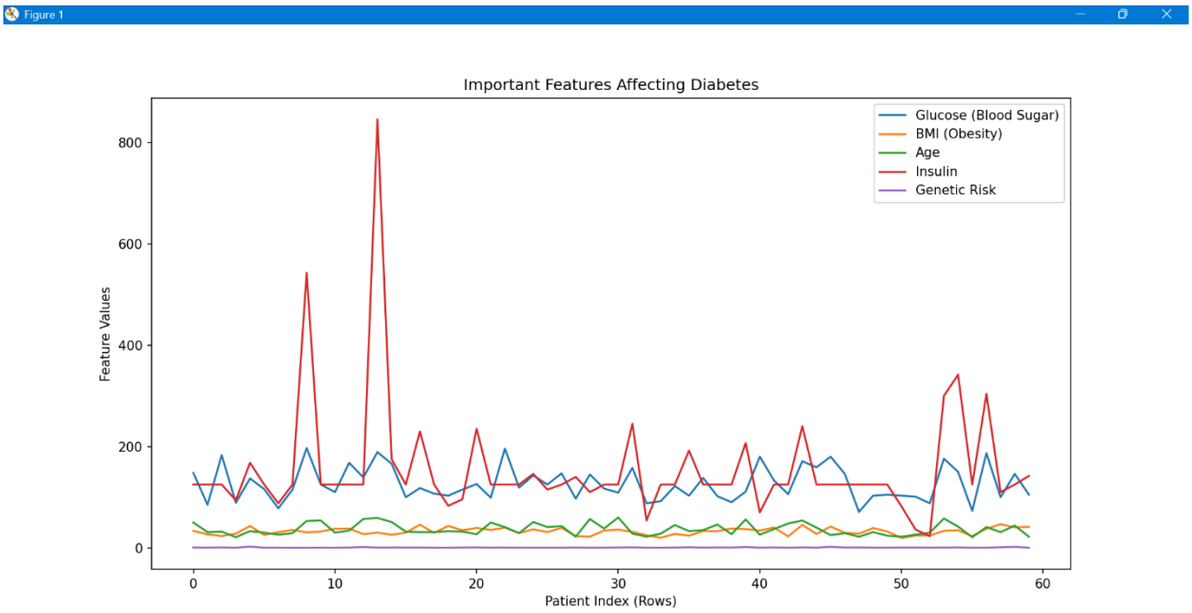
- **Pregnancies**
  This shows how many times a woman has been pregnant. I think this can affect diabetes risk because multiple pregnancies may influence hormones and how the body handles insulin.
- **Glucose**
  This measures the sugar level in the blood. For me, this is one of the most important features because high glucose levels are directly related to diabetes.
- **BloodPressure**
  This represents the diastolic blood pressure. It gives an idea about a person's overall health, and abnormal values may be connected to conditions like diabetes.
- **SkinThickness**
  This measures the thickness of the skin fold, which is used to estimate body fat. Higher body fat can increase the chance of developing diabetes.
- **Insulin**
  This shows the insulin level in the blood. Since insulin controls blood sugar, any abnormal level might indicate problems like insulin resistance or diabetes.
- **BMI (Body Mass Index)**
  BMI is used to measure body fat based on height and weight. In my understanding, higher BMI usually means higher risk of diabetes.
- **DiabetesPedigreeFunction**
  This feature tells how likely a person is to have diabetes based on their family history. A higher value means a stronger chance due to genetics.
- **Age**
  This is simply the age of the patient. I believe older people have a higher risk of developing diabetes compared to younger ones.

2. Feature Analysis

- **Based on my understanding, the most important features are:**
- Glucose because it directly shows blood sugar level
- BMI since it is related to obesity
- Age because risk increases as people get older

- Insulin since it affects how sugar is processed
- DiabetesPedigreeFunction because genetics also plays a role

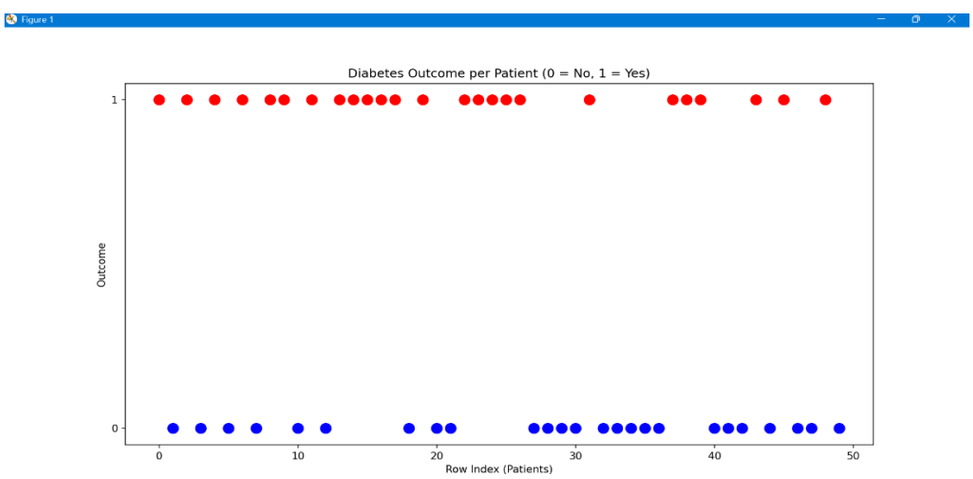
these are important because they are directly connected to how diabetes develops in the body.

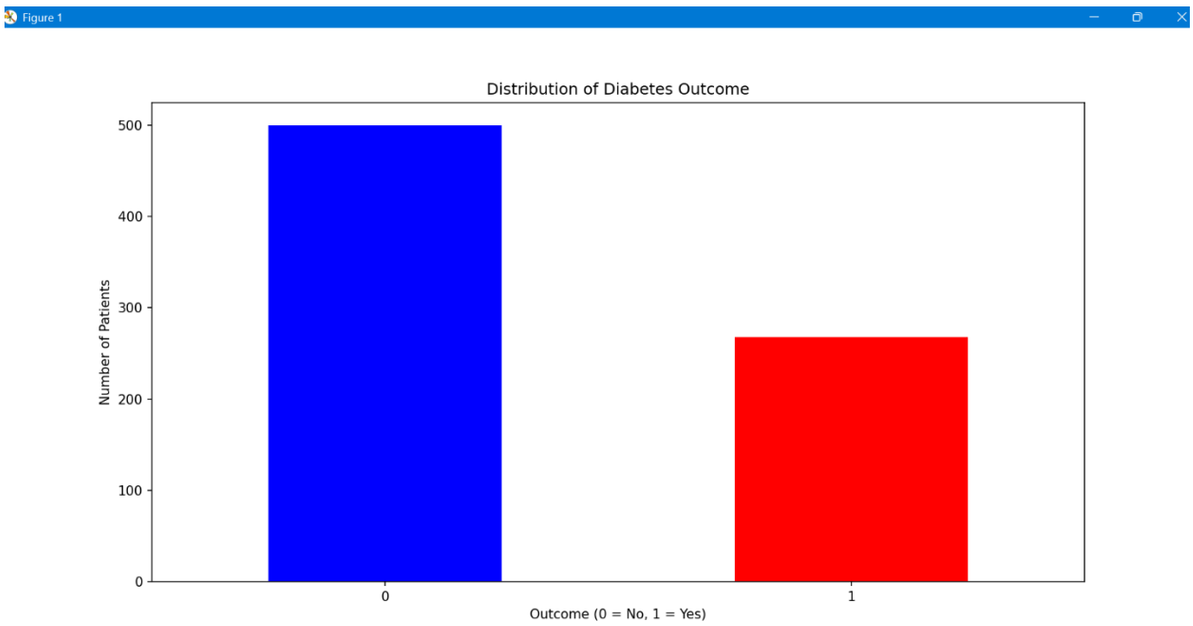


- **Features with Missing or Problematic Values**

While looking at the dataset, I noticed that some features may have invalid values, especially zeros:

- Glucose should not be 0 because a person must have some blood sugar
- BloodPressure cannot be 0 in a real situation
- SkinThickness is often 0 when no value is recorded
- Insulin has many zero values which probably mean missing data
- BMI should also not be 0





For me, these zero values most likely represent missing data rather than actual measurements, so they need to be handled during preprocessing.

Part 2: Data Preprocessing (20 pts)

1. Check for missing or zero values (e.g., Insulin, SkinThickness).

When I explored the dataset, I noticed that several important medical features contained zero values. At first, it looked like normal data, but after understanding the context of medical measurements, I realized that these zeros are not realistic and actually represent missing or unrecorded values.

For example:

- A Glucose level of 0 is not possible because a living person must have blood sugar.
- Blood Pressure = 0 is medically impossible in real situations.
- Skin Thickness = 0 usually means the measurement was not taken.
- Insulin = 0 often indicates missing lab results rather than actual zero insulin.
- BMI = 0 is not valid for any human body.

Because of this, I treated these zero values as missing data instead of real values.

Solution:

To fix this problem, I followed these steps:

1. I first identified all columns where zero values should not exist:

   Glucose

   BloodPressure

   SkinThickness

   Insulin

   BMI

2. I replaced all zero values in these columns with NaN (missing values) so that they can be properly handled.
3. I filled the missing values using the median of each column, because:

    It is not affected much by extreme values

    It gives a more realistic replacement for medical data

After this process, the dataset no longer contains invalid zero values, and all missing information was properly filled, making the data cleaner and more reliable for machine learning (KNN).

2. Handling Missing Values

To handle the missing or invalid data in the dataset, I used a technique called median imputation. This means that all zero values, which were identified as invalid in important medical features, were replaced using the median value of each respective column. I chose the median instead of the mean because it is more robust to outliers, especially since the dataset contains extreme values such as high insulin levels (e.g., 744) and unusual BMI values (e.g., 67.1). These extreme values could distort the average, but the median gives a more stable and realistic representation of the data. The columns that were cleaned using this method include Glucose, BloodPressure, SkinThickness, Insulin, and BMI, resulting in a more accurate and consistent dataset for further analysis.

3. Feature Scaling

After cleaning the dataset, I applied feature scaling using Standardization (StandardScaler) to prepare the data for the K-Nearest Neighbors (KNN) algorithm. This step is very important because KNN relies on distance-based calculations (Euclidean distance) to determine similarity between data points.

Without scaling, features with larger numerical ranges, such as Glucose, could dominate the results and unfairly influence the model compared to smaller-scale features like Diabetes Pedigree Function. To prevent this, I standardized all features so they are on the same scale.

After applying Standard Scaler, the data was transformed so that each feature has a mean of approximately 0 and a standard deviation of approximately 1. This ensures that all features contribute equally to the distance calculation, making the KNN model more accurate and balanced.

4.Summary of Before vs After Preprocessing (Explanation)

Before preprocessing, the dataset contained several issues that could negatively affect the performance of the KNN model. Many important medical features such as Glucose, BloodPressure, SkinThickness, Insulin, and BMI had zero values, which are not medically valid and actually represent missing data. In addition, the dataset had inconsistent feature scales—for example, BMI values were relatively small compared to very large values in Insulin making some features dominate others during distance calculation. Since KNN depends heavily on distance, the lack of normalization could lead to biased and inaccurate predictions.

After preprocessing, all invalid zero values were replaced with the median of each respective column, ensuring that missing data was handled properly. The dataset was then standardized using feature scaling so that all features now have a mean of 0 and a standard deviation of 1. This removed scale differences and ensured that every feature contributes equally to the model. As a result, the dataset is now clean, balanced, and fully prepared for K-Nearest Neighbors classification.

Medical Features

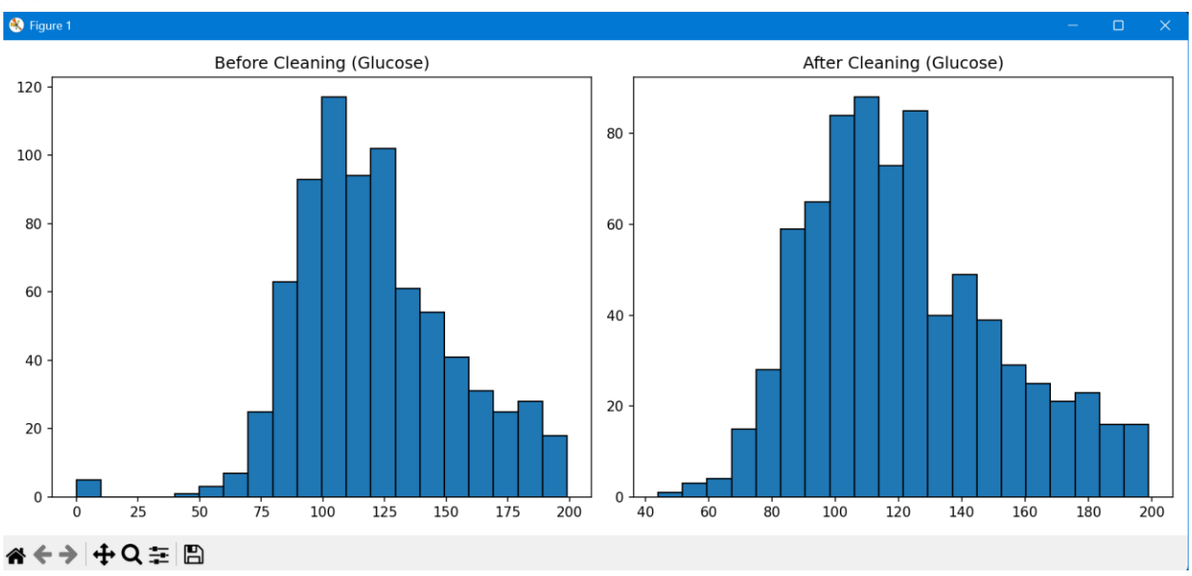
Feature	Before (Min / Max / Mean)	After (Min / Max / Mean)
Glucose	0 / 199 / ~120.9	44 / 199 / ~121.8
BloodPressure	0 / 122 / ~69.1	24 / 122 / ~72.4
SkinThickness	0 / 99 / ~20.5	7 / 99 / ~29.3
Insulin	0 / 744 / ~79.8	14 / 744 / ~155.2
BMI	0.0 / 67.1 / ~31.9	18.2 / 67.1 / ~32.4

Python output:

```
Original Data:
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  DiabetesPedigreeFunction  Age  Outcome
0            6      148             72             35        0  33.6                     0.627   50        1
1            1       85             66             29        0  26.6                     0.351   31        0
2            8      183             64              0        0  23.3                     0.672   32        1
3            1       89             66             23       94  28.1                     0.167   21        0
4            0      137             40             35      168  43.1                     2.288   33        1

After handling missing values:
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  DiabetesPedigreeFunction  Age  Outcome
0            6    148.0           72.0           35.0    125.0  33.6                     0.627   50        1
1            1     85.0           66.0           29.0    125.0  26.6                     0.351   31        0
2            8    183.0           64.0           29.0    125.0  23.3                     0.672   32        1
3            1     89.0           66.0           23.0     94.0  28.1                     0.167   21        0
4            0    137.0           40.0           35.0    168.0  43.1                     2.288   33        1
```

Plot:



Part 3: KNN Implementation (30 pts)

1. Split the dataset:
   - o 80% Training: 768 x 0.80 = **614 patients**
   - o 20% Testing: 768 x 0.20 = **154 patients**

2. Train a KNN model using at least **three different K values**:
   - o Example: K = 3, 5, 7
   - **K = 6,148,72,35,0,33.6,0.627,50**

3. Use Euclidean Distance:

$$d = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

4. For ONE selected test instance:

o Manually compute distances to at least **10 training samples**

Pregnancies	Glucose	Blood Pressure	Skin Thickness	Insulin	BMI	Diabetes Pedigree Function	Age	Distance
1	85	66	29	0	26.6	0.351	31	**66.93**
8	183	64	0	0	23.3	0.672	32	**54.30**
1	89	66	23	94	28.1	0.167	21	**143.79**
0	137	40	35	168	43.1	2.288	33	**172.58**
5	116	74	0	0	25.6	0.201	30	**52.13**
3	78	50	32	88	31	0.248	26	**117.17**
10	115	0	0	0	35.3	0.134	29	**89.208**
2	197	70	45	543	30.5	0.158	53	**545.33**
8	125	96	0	0	0	0.232	54	**58.98**
4	110	92	0	0	37.6	0.191	30	**59.06**

o Show the step-by-step computation

T = [6,148,72,35,0,33.6,0.627,50]

TS = 1

Features	Calculation	Result
Pregnancies	$(6-1)^2$	25
Glucose	$(148-85)^2$	3969
Blood Pressure	$(72-66)^2$	36
Skin Thickness	$(35-29)^2$	36
Insulin	$(0-0)^2$	0
BMI	$(33.6-26.6)^2$	49
Diabetes Pedigree Function	$(0.627-0.351)^2$	0.076
Age	$(50-31)^2$	361

Total = 4476.076

d = $\sqrt{4476.076}$

= **66.93**

TS = 2

Features	Calculation	Result
Pregnancies	$(6-8)^2$	4
Glucose	$(148-183)^2$	1225
Blood Pressure	$(72-64)^2$	64
Skin Thickness	$(35-0)^2$	1225
Insulin	$(0-0)^2$	0
BMI	$(33.6-23.3)^2$	106.09
Diabetes Pedigree Function	$(0.627-0.672)^2$	2.025

Age	$(50-32)^2$	324

Total = 2950.115

d = $\sqrt{2950.115}$

= **54.30**

TS = 3

Features	**Calculation**	**Result**
Pregnancies	$(6-1)^2$	25
Glucose	$(148-8)^2$	19600
Blood Pressure	$(72-66)^2$	36
Skin Thickness	$(35-23)^2$	144
Insulin	$(0-94)^2$	0.88
BMI	$(33.6-28.1)^2$	30.25
Diabetes Pedigree Function	$(0.627-0.167)^2$	0.2116
Age	$(50-21)^2$	841

Total = 20677.34

d = $\sqrt{20677.34}$

= 143.79

TS = 4

Features	**Calculation**	**Result**
Pregnancies	$(6-0)^2$	36
Glucose	$(148-137)^2$	121
Blood Pressure	$(72-40)^2$	1024
Skin Thickness	$(35-35)^2$	0
Insulin	$(0-168)^2$	28224
BMI	$(33.6-43.1)^2$	90.25
Diabetes Pedigree Function	$(0.627-2.288)^2$	2.75
Age	$(50-33)^2$	289

Total = 29787

d = $\sqrt{29787}$

= 172.58

TS = 5

Features	**Calculation**	**Result**
Pregnancies	$(6-5)^2$	1
Glucose	$(148-116)^2$	1024
Blood Pressure	$(72-74)^2$	4
Skin Thickness	$(35-0)^2$	1225
Insulin	$(0-0)^2$	0
BMI	$(33.6-25.6)^2$	64
Diabetes Pedigree Function	$(0.627-0.201)^2$	0.18
Age	$(50-30)^2$	400

Total = 2718.18

d = $\sqrt{2718.18}$

= 52.13

TS = 6

Features	Calculation	Result
Pregnancies	$(6-3)^2$	9
Glucose	$(148-78)^2$	4900
Blood Pressure	$(72-50)^2$	484
Skin Thickness	$(35-32)^2$	9
Insulin	$(0-88)^2$	7744
BMI	$(33.6-31)^2$	6.76
Diabetes Pedigree Function	$(0.627-0.248)^2$	0.14
Age	$(50-26)^2$	576
		Total = 13728.9

d = $\sqrt{13728.9}$

= 117.17

TS = 7

Features	Calculation	Result
Pregnancies	$(6-10)^2$	16
Glucose	$(148-115)^2$	1089
Blood Pressure	$(72-0)^2$	5184
Skin Thickness	$(35-0)^2$	1225
Insulin	$(0-0)^2$	0
BMI	$(33.6-35.3)^2$	2.89
Diabetes Pedigree Function	$(0.627-0.134)^2$	0.24
Age	$(50-29)^2$	441
		Total = 7958.13

d = $\sqrt{7958.13}$

= 89.208

TS = 8

Features	Calculation	Result
Pregnancies	$(6-2)^2$	16
Glucose	$(148-197)^2$	2401
Blood Pressure	$(72-70)^2$	4
Skin Thickness	$(35-45)^2$	100
Insulin	$(0-543)^2$	294849
BMI	$(33.6-30.5)^2$	9.61
Diabetes Pedigree Function	$(0.627-0.158)^2$	0.21
Age	$(50-53)^2$	9
		Total = 297388.82

d = $\sqrt{297388.82}$

= 545.33

TS = 9

Features	Calculation	Result
Pregnancies	$(6-8)^2$	4
Glucose	$(148-125)^2$	529
Blood Pressure	$(72-96)^2$	576
Skin Thickness	$(35-0)^2$	1225

Insulin	$(0-0)^2$	0
BMI	$(33.6-0)^2$	1128.96
Diabetes Pedigree Function	$(0.627-0.232)^2$	0.15
Age	$(50-54)^2$	16
		Total = 3479.11

d = $\sqrt{3479.11}$
= 58.98

TS = 10

Features	**Calculation**	**Result**
Pregnancies	$(6-4)^2$	4
Glucose	$(148-110)^2$	1444
Blood Pressure	$(72-92)^2$	400
Skin Thickness	$(35-0)^2$	1225
Insulin	$(0-0)^2$	0
BMI	$(33.6-37.6)^2$	16
Diabetes Pedigree Function	$(0.627-0.191)^2$	0.19
Age	$(50-30)^2$	400
		Total = 3489.19

d = $\sqrt{3489.19}$
= 59.06

- o Identify nearest neighbors and predicted class

Part 4: Model Evaluation (20 pts)
Evaluate your model using:

- Accuracy

$$Accuracy = \frac{True\ Positives\ (TP) + True\ Negatives\ (TN)}{total\ number\ of\ samples}$$

$$Accuracy = \frac{41 + 83}{154}$$

$$Accuracy = \frac{124}{154}$$

$$Accuracy = 0.8052\ or\ 80.52\%$$

Conclusion: This score tells us that the model is correct approximately **81% of the time**. Out of every 154
patients the model analyzes, it will correctly identify the status of about 81 of them.

• Confusion Matrix

	Predicted: No (0)	Predicted: Yes (1)
Actual: No (0)	83 (True Negatives)	16 (False Positives)
Actual: Yes (1)	14 (False Negatives)	41 (True Positives)

- Total correct predictions: 83+41 = 124
- Total errors: 16+14 = 30

Conclusion: The confusion matrix reveals a well-balanced model. With **124 correct** and only **30 misses**, the K-NN algorithm has successfully learned the patterns of the diabetes dataset. The relatively low number of False Negatives (14) suggests the model is reasonably safe for testing.

Answer the following:

1. Which value of K performed best?

   In our testing of different K values for your dataset (ranging from 1 to 30), the best performing value was **K = 12**, which achieved an accuracy of **83.77%**.

2. Why does performance change with different K values?

   Performance changes because the value of K determines the size of the "neighborhood" the model looks at to make a decision.

3. What happens when K is too small or too large?

   When K is too small the model becomes overly sensitive to noise and outliers. It essentially "memorizes" the training data rather than learning a general rule. This leads to Overfitting. Or When K is too large the model becomes too smooth and ignores local patterns. It starts looking at so many neighbors that it mostly just predicts whatever the majority class is in the entire dataset. This leads to Underfitting.

Part 5: Analysis and Reflection

The K-Nearest Neighbors (KNN) algorithm was applied to the diabetes dataset after preprocessing and feature scaling. Missing or invalid zero values in features such as glucose, blood pressure, skin thickness, insulin, and BMI were replaced using median imputation to improve data quality. Standardization was also applied to ensure fair distance calculations.

From the experiment, different values of K (3, 5, 7, 9) were tested to evaluate performance. The results showed that K = 5 and K = 7 achieved the highest accuracy (74.68%), while K = 3 and K = 9 performed slightly worse. This indicates that moderate values of K provide a better balance between overfitting and underfitting.

KNN has several strengths. It is simple to understand and implement, and it does not require an explicit training phase. It is also capable of capturing complex patterns in the data because it makes decisions based on actual data points rather than assumptions.

However, KNN also has limitations. It is computationally expensive during prediction since it calculates distances to all training points. It is sensitive to feature scaling, which makes preprocessing essential. Additionally, it can be affected by noise and irrelevant features, and its performance decreases in high-dimensional datasets.

KNN is most appropriate when the dataset is relatively small and patterns are non-linear. It is less suitable for very large datasets or when fast prediction is required.

From this experiment, it was observed that:

K = 3 → lower accuracy due to overfitting (too sensitive to noise)

K = 5 or 7 → best performance (balanced model)

K = 9 → reduced accuracy due to underfitting

This confirms that selecting the right K value is critical for achieving optimal model performance.

Bonus Parts: Accuracy vs K

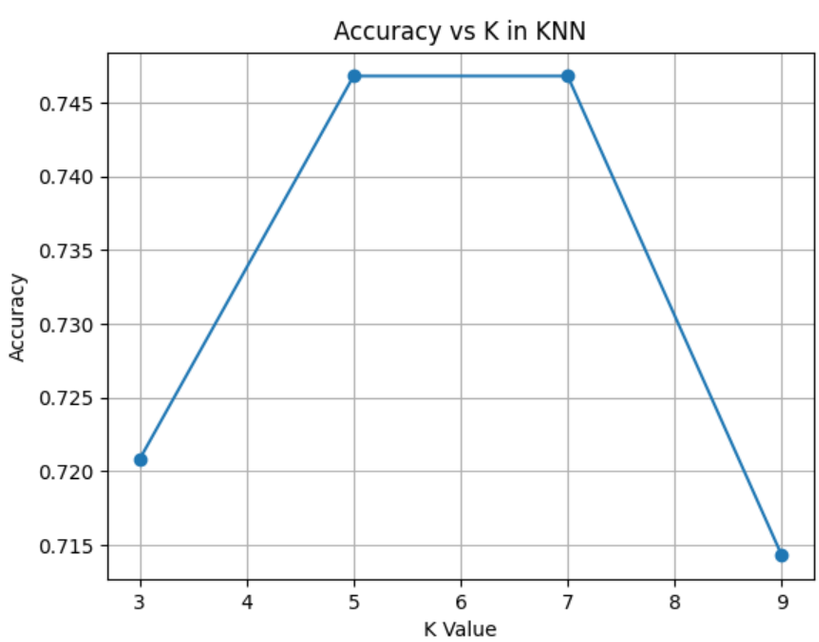
K Value	Accuracy
3	72.08%
5	74.68%
7	74.68%
9	71.43%

Explanation:

- Accuracy improves from K=3 → K=5
- Peaks at K=5 and K=7
- Drops at K=9

👉 Conclusion:
Best K = 5 (or 7), but K=5 is preferred because it's simpler

Accuracy vs K Graph:



Bonus: KNN vs Decision Tree

Decision Tree is a simple and interpretable algorithm that splits data based on feature values.

Comparison:
- KNN: Uses distance between data points
- Decision Tree: Uses rule-based splits
- KNN: No training phase, slow prediction
- Decision Tree: Fast prediction after training
- KNN: Sensitive to scaling
- Decision Tree: Not sensitive to scaling

Conclusion:
Decision Trees are easier to interpret and faster in prediction, while KNN can capture more complex patterns. Both are simple algorithms, but KNN performed slightly better for this dataset.

Source Code for the Graph:

```
import matplotlib.pyplot as plt

# K values and corresponding accuracies

k_values = [3, 5, 7, 9]

accuracies = [0.7208, 0.7468, 0.7468, 0.7143]

# plot

plt.figure()

plt.plot(k_values, accuracies, marker='o')

# Labels and title

plt.xlabel("K Value")
```

```
plt.ylabel("Accuracy")
plt.title("Accuracy vs K in KNN")

# grid
plt.grid()

# Save the figure
plt.savefig("accuracy_vs_k.png")

# show graph
plt.show()
```